CCA-F: Points and Terms by Topic

Every topic in the course, grouped by domain. For each: the key points to memorise, then the terms to know. Pair it with the shorter Memorisation Sheet for final review.

Domain 1: Agentic architecture

1. Agents, workflows and chatbots

KEY POINTS

- A **chatbot** answers; a **workflow** follows code paths you predefined; an **agent** decides its own next step and tools in a loop.
- Default to the **simplest architecture that works**: single call -> workflow -> agent.
- Workflows win when the steps are **known and stable**: cheaper, faster, predictable, easy to test.
- Agents earn their cost only when the path is **open-ended** and can't be scripted in advance.

TERMS

Workflow	Predefined, code-controlled sequence of LLM calls and tools.
Agent	Model-directed loop: it chooses actions until the goal is met.
Five workflow patterns	Prompt chaining, routing, parallelisation, orchestrator-workers, evaluator-optimizer.

2. The agent loop

KEY POINTS

- Cycle: **perceive -> reason -> act (tool call) -> observe (tool result)** -> repeat.
- The model can reason but can't act; **tools are the foundation of agency.**
- Continue while `stop_reason` is `"tool_use"`; stop on `"end_turn"`.
- **Never detect completion by parsing the model's text.**

TERMS

Observation	The tool result fed back to the model as the next input.
`stop_reason`	API field saying why the model stopped: tool_use, end_turn, max_tokens, refusal, etc.

3. Loop control

KEY POINTS

- Every loop needs **hard caps**: max iterations, token/cost budget, timeout.
- Stop conditions live in **code**, not in the prompt.
- Three exits: **finish** (goal met), **stop** (budget hit), **escalate** (needs a human).
- Repeated identical failures = stop and escalate, don't loop forever.

TERMS

Iteration cap	Maximum number of loop turns before forced stop.
Escalation	Handing control to a human or higher-level system.

4. Decomposition

KEY POINTS

- Break a goal into subtasks that are **specific, verifiable and appropriately sized.**
- Three shapes: **hierarchical** (tree of subgoals), **sequential** (step depends on prior), **parallel** (independent).
- Dependencies decide the shape: dependent -> sequential; independent -> parallel.

TERMS

Hierarchical decomposition	Goal split into subgoals, each split further.
Dependency	Step B needs step A's output, so B must wait.

5. Pipelines and parallel execution

KEY POINTS

- **Sequential pipeline**: each stage feeds the next; simple to debug, but latency adds up and errors propagate.
- Validate between stages so a bad output doesn't cascade.
- **Fan-out** independent work in parallel, **fan-in** to aggregate; needs a sync point and a policy for partial failures.

TERMS

Fan-out	Dispatching independent subtasks at the same time.
Fan-in	Collecting and merging the parallel results.
Synchronisation	Waiting point where all branches must report before continuing.

6. Adaptive planning and ambiguity

KEY POINTS

- Plans should **update mid-execution** when observations contradict assumptions.
- For ambiguous specs: **ask a clarifying question** when the ambiguity changes the outcome; otherwise state an assumption and proceed.
- Don't silently guess on high-impact ambiguity.

TERMS

Replanning	Revising the remaining plan based on new information.
Assumption logging	Recording guesses made so they can be checked later.

7. Orchestrators and subagents

KEY POINTS

- **Orchestrator**: directs, delegates, aggregates. It owns the overall plan.
- **Subagents** get a narrow scope, limited tools (least privilege) and an **isolated context.**
- Subagents **inherit nothing**: the brief must contain all needed context.
- They return a **summary**; anything not returned is lost.

TERMS

Context isolation	Subagent's messy work stays out of the parent's context.
Task brief	The explicit instructions and context passed to a subagent.

8. Topologies and cost

KEY POINTS

- **Hub-and-spoke**: central orchestrator, most common, easy to control.
- **Pipeline**: fixed sequence of specialised agents.
- **Peer-to-peer**: agents talk directly; flexible but hard to debug and control.
- Cost: **~4x tokens** single agent with tools, **~15x** multi-agent. Use multi-agent only for high-value, parallelisable work.

TERMS

Hub-and-spoke	One coordinator, many workers that report only to it.
Peer-to-peer	Agents communicate laterally without a central coordinator.

9. Handoffs, schemas, state and sessions

KEY POINTS

- Agent-to-agent messages use **structured schemas** (typed fields), not free text.
- A handoff carries: goal, context so far, what was done, what's left.
- **In-context state** is fast but lost on truncation; **external memory** survives sessions.
- Session continuity: persist checkpoints externally so work can **resume after failure.**

TERMS

In-context state	Everything currently in the model's context window.
External memory	State in databases, files or vector stores outside the model.
Semantic retrieval	Querying memory by meaning via embeddings, not exact keys.

10. Errors, retries and fallbacks

KEY POINTS

- Three error types: **tool errors**, **reasoning errors**, **environment errors.**
- Detect failures **early**, before they cascade.
- **Transient** -> retry with backoff; **permanent** (bad input, auth) -> don't retry.
- Fallback chain: primary tool -> alternative -> cached result -> escalate.
- **Graceful degradation** signals reduced quality; **silent degradation** hides it (bad).

TERMS

Exponential backoff	Waiting progressively longer between retries.
Fallback chain	Pre-planned ordered list of alternatives when the primary fails.
Graceful degradation	Returning a partial result with an explicit quality flag.

11. Enforcement, escalation and interruption points

KEY POINTS

- **Prompts are probabilistic**, so they're not enough for high-stakes paths.
- **Programmatic enforcement** (code, hooks, validation) for anything that must never happen.
- Valid triggers are **explicit conditions**: policy exception, irreversible or high-value action, retry budget exhausted, customer asks for a human.
- Weak triggers: vague "when unsure", sentiment alone, model's self-reported confidence alone.
- Put human **interruption points** before irreversible actions; avoid over-escalation (reviewer fatigue).

TERMS

HITL	Human-in-the-loop: human checkpoint in the execution path.
Programmatic enforcement	Constraints checked in code, independent of model output.
Interruption point	Planned pause where a human approves before continuing.

Domain 2: Tool design and MCP

1. How Claude selects tools

KEY POINTS

- A tool = **name + description + input schema.** The model never sees your code.
- Selection is **semantic matching on the description**, so tool design is prompt engineering.
- Overlapping or vague descriptions cause **non-deterministic mis-routing.**

TERMS

Tool definition	Name, description and JSON schema sent to the model.
Mis-routing	Model calls the wrong tool because descriptions overlap.

2. Descriptions and schemas

KEY POINTS

- Description states **purpose, when to use, when NOT to use, side effects.**
- Schema: clear field names, types, **required vs optional**, per-field descriptions.
- Use **enums** for fixed choices and sensible defaults: make tools hard to misuse.

TERMS

Required parameter	Must be supplied or the call is invalid.
Enum	Fixed list of allowed values for a field.

3. Strict tool use

KEY POINTS

- `strict: true` applies **constrained decoding** to tool arguments.
- Guarantees arguments match the schema: correct types, no invented fields.

TERMS

Constrained decoding	Generation restricted so only schema-valid tokens can be produced.

4. Tool errors and the silent false negative

KEY POINTS

- Never fail silently. Return a **typed status: success / empty / partial / access-failure.**
- **Empty** = searched and found nothing (valid). **Access failure** = never reached the data.
- Wrapper rules: empty -> no retry; access failure -> retry up to a limit, then escalate.
- Partial success: report what succeeded so retries don't duplicate work.

TERMS

Silent false negative	Agent says "nothing exists" because a tool failed silently.
Tool wrapper	Thin layer that normalises output and attaches a status.
Status enum	success, empty, partial, access_failure.

5. Tool scoping and selection control

KEY POINTS

- Give each agent **only the tools it needs** (less is more).
- Accuracy degrades past **~30-50 tools**; each definition costs tokens.
- `tool_choice`: **auto** (model decides), **any** (must use a tool), **tool** (must use this one), **none.**
- Namespace related tools to avoid clashes.

TERMS

Least privilege	Minimum tools and access required for the job.
Namespacing	Prefixing names, e.g. github_create_issue.

6. Tool search for large catalogs

KEY POINTS

- Large catalogs hit an **accuracy cliff** and burn context (tens of thousands of tokens of definitions).
- **Tool search tool +** `defer_loading`: definitions load only when searched for.

TERMS

Deferred loading	Tool definition kept out of context until needed.
Tool search tool	Lets Claude find relevant tools on demand.

7. Built-in vs custom tools

KEY POINTS

- **Server tools** (web search, code execution) run on Anthropic's infrastructure.
- **Client tools** run in your app; the model pauses and waits for your result.
- **Computer use** is an Anthropic-schema client tool; sandboxing it is your responsibility.
- Build custom tools only when built-ins can't do the job.

TERMS

Server tool	Executed by Anthropic within the API turn.
Client tool	Executed by your application; MCP tools are client-side.

8. MCP architecture and primitives

KEY POINTS

- MCP = open standard connecting AI apps to tools and data.
- **Client = AI app (brain). Server = capability provider (hands)**, usually no AI.
- **Tools = model-controlled, resources = app-controlled, prompts = user-controlled.**
- Design question: **who should decide when this happens?**

TERMS

MCP host/client	The AI application that connects to servers.
Resource	Read-only data the app exposes (files, records).
Prompt template	Reusable prompt the user picks, e.g. a slash command.

9. Transports

KEY POINTS

- **STDIO**: standard input/output to a local process; single user.
- **Streamable HTTP**: remote/networked, multi-user, production.
- SSE is **not** a standalone transport; MCP doesn't use WebSocket.

TERMS

STDIO	Client launches server as a subprocess and talks over stdin/stdout.
Streamable HTTP	HTTP transport for remote servers, can stream responses.

10. Sampling, roots, notifications

KEY POINTS

- **Sampling**: server asks the client to run an LLM call; needs **user approval.**
- **Roots**: client advises which paths to work in; **advisory, not enforced.**
- **Progress notifications** (progress + total) and **log notifications** (severity) for long tasks.

TERMS

Sampling	"Lend me your brain": server -> client LLM request.
Roots	"Work over here": client -> server scope hint.

11. Security and production

KEY POINTS

- Each server is a **trust boundary**; vet before connecting.
- **Prompt injection via tool results**: treat external content as data, not commands.
- Real enforcement: OS permissions, sandboxing, deny rules, least privilege.
- Remote servers: **OAuth 2.1** recommended for auth; **semantic versioning**; **rate limiting.**

TERMS

Prompt injection	Malicious instructions hidden in content the model reads.
Semantic versioning	Major = breaking, minor = compatible feature, patch = fix.

Domain 3: Claude Code

1. What Claude Code is; tools

KEY POINTS

- Agentic coding tool in the terminal, IDE, desktop, web.
- Core tools: **Read, Edit/Write, Bash, Grep/Glob, web search/fetch, Task/Agent (subagents).**
- Everything read or run in a session **consumes context.**

TERMS

Agentic loop	Claude reads, edits, runs, observes, repeats until done.

2. Onboarding and context

KEY POINTS

- `/init` generates a starter CLAUDE.md from the codebase.
- Reference files with `@path`; keep context focused; `/clear` between unrelated tasks; `/compact` to summarise.

TERMS

@-mention	Pulls a specific file into context.

3. CLAUDE.md hierarchy

KEY POINTS

- Scopes: **managed policy, user (**`~/.claude/CLAUDE.md`**), project (repo root), local (**`CLAUDE.local.md`**).**
- Files are **concatenated**; genuine contradictions resolve **non-deterministically.**
- **Subdirectory CLAUDE.md lazy-loads and augments** when working there.
- Write short, specific, actionable rules; avoid vague or conflicting ones.

TERMS

CLAUDE.local.md	Personal, gitignored project instructions (about people).
Subdirectory CLAUDE.md	Folder-scoped rules (about place).

4. Settings and permissions

KEY POINTS

- Settings precedence (deterministic): **managed -> CLI args ->** `.claude/settings.local.json` **->** `.claude/settings.json` **->** `~/.claude/settings.json`**.**
- Modes: **default, acceptEdits, plan, bypassPermissions**; toggle with **Shift+Tab.**
- Exclude files with a **Read deny rule**; `.claudeignore` doesn't exist; `.gitignore` doesn't block reads.

TERMS

Allow/deny/ask rules	Built-in permission rules per tool and path.
bypassPermissions	No prompts; only for sandboxed environments.

5. Slash commands and skills

KEY POINTS

- Commands: Markdown in `.claude/commands/`; filename = name; `$ARGUMENTS`, `$1`, `$2`.
- Commit to the repo to share with the team.
- **Skill** = `SKILL.md` + optional files; **auto-loads when its description matches** the request.
- Stack: CLAUDE.md always on -> skills on match -> commands on demand.

TERMS

Skill	Reusable instruction package loaded by description match.
Slash command	Manually invoked prompt template.

6. Subagents

KEY POINTS

- Defined as Markdown with frontmatter in `.claude/agents/`: name, description, tools, model.
- Own context, inherit nothing; main benefit = **context isolation.**
- Precise prompt + defined return shape; treat returns as hypotheses to verify.

TERMS

Custom subagent	Named specialist Claude delegates to via its description.

7. Plan mode

KEY POINTS

- **Read-only** phase: explores and proposes an ordered plan; writes nothing.
- **Approval gate**: accept, edit or reject. **Replan** if execution reveals a flaw.
- Use for multi-file, risky or unfamiliar changes.

TERMS

Approval gate	Checkpoint before any file is modified.

8. Hooks

KEY POINTS

- Scripts that run at **lifecycle events**, outside Claude's context; Claude can't override them.
- Common events: **PreToolUse, PostToolUse, UserPromptSubmit, Stop** (plus SessionStart, Notification and more).
- Order: **PreToolUse -> permission check -> tool -> PostToolUse.**
- Exit codes: **0 continue, 2 block (stderr goes to Claude), other non-zero non-blocking.**
- Uses: block dangerous commands, auto-format/lint after edits, logging.

TERMS

PreToolUse	Fires before a tool runs; can block it.
PostToolUse	Fires after; can't undo the action, can react to it.

9. CI/CD and the Agent SDK

KEY POINTS

- `claude -p "..."` = **headless / non-interactive**; pre-authorise with scoped allow/deny rules.
- Structured output flags (e.g. `--output-format json`) for pipelines.
- **Agent SDK** (formerly Claude Code SDK): drive sessions from code, stream tool events, resume sessions.

TERMS

Headless mode	Runs without an interactive user, e.g. in CI.
Agent SDK	Programmatic interface to Claude Code's agent loop.

10. MCP in Claude Code

KEY POINTS

- Add servers with `claude mcp add`; scopes local, project (`.mcp.json`, shared) and user.
- Same primitives and transports as Domain 2.

TERMS

`.mcp.json`	Project-level MCP config committed for the team.

Domain 4: Prompt engineering and structured output

1. Clarity and prompt anatomy

KEY POINTS

- Building blocks: **role, context, task, output format, constraints, examples.**
- **Ambiguity causes variance**; specific instructions reduce it.
- Decompose complex instructions into explicit steps and constraints.

TERMS

Constraint	An explicit rule the output must obey.

2. System prompt, roles, XML

KEY POINTS

- **Standing rules -> system prompt; one-off requests -> user turn.**
- Role/persona sets expertise, tone and audience.
- Use **XML tags** (`<document>`, `<instructions>`) to separate parts; put long documents first, question last.

TERMS

System prompt	Persistent instructions for the whole conversation.
XML tags	Labels that delimit sections of a prompt.

3. Few-shot examples

KEY POINTS

- **Few-shot = multishot.** ~3-5 examples, **diverse**, covering edge cases.
- Examples anchor format strongly; if they contradict instructions, **examples win.**

TERMS

Format anchoring	Model copies the structure shown in examples.

4. Chain-of-thought and extended thinking

KEY POINTS

- **CoT** = prompt technique; separate reasoning and answer with tags.
- **Extended thinking** = API feature; separate billed thinking blocks; effort levels.
- Ladder: plain prompt -> CoT -> extended thinking.

TERMS

Thinking block	Separate reasoning content returned by extended thinking.

5. Steering and sampling

KEY POINTS

- **Prefilling returns HTTP 400 on 4.6+ models**; use structured outputs, strict tools or system-prompt steering.
- Sampling knobs (temperature, top_p, top_k) are **legacy** on current models (non-default values rejected); steer with prompting and effort.
- Steering anti-patterns: over-constraining, conflicting signals, redundancy.

TERMS

Temperature	Legacy: lower = focused, higher = varied.
top_p	Legacy nucleus sampling by cumulative probability.

6. Multi-turn interactions

KEY POINTS

- Every turn re-sends history; context grows each turn.
- Keep state explicit; restate key constraints rather than relying on early turns.

TERMS

Conversation state	Accumulated messages the model sees each call.

7. Structured outputs

KEY POINTS

- **Schema + constrained decoding = guaranteed valid JSON.** Asking in the prompt is not a guarantee.
- Works for a **supported subset** of JSON Schema; SDK moves unsupported keywords into descriptions and validates after.
- Guarantee gaps: **refusal** and **max_tokens truncation**; always check `stop_reason`.
- Tool definitions can also be used to extract structured data.

TERMS

Enforceable subset	Schema features the grammar engine can guarantee.
Guarantee gap	Cases (refusal, truncation) where output may not match.

8. Schema design for extraction

KEY POINTS

- Make uncertain fields **nullable/optional** so the model doesn't invent values.
- Enums with an "other"/"unknown" option; per-field descriptions.

TERMS

Nullable field	Allows null instead of forcing a fabricated value.

9. Validation, retries and review

KEY POINTS

- Validate -> on failure **retry with the specific error fed back**, with a retry cap.
- Semantic checks beyond schema (totals add up, dates valid).
- **Multi-pass review**: second call critiques; **self-review** is biased, **cross-review** is more independent.

TERMS

Validation gate	Check that output must pass before moving on.
Reviewer pattern	Separate model call that only critiques.

10. Evaluation

KEY POINTS

- Test dataset: handcrafted, synthetic and production cases incl. edge and adversarial.
- **Code-based grading**: fast, deterministic. **Model-based grading**: scales to subjective tasks; calibrate against humans.

TERMS

Eval	Systematic test of a prompt against a dataset.
Model-based grading	LLM scores outputs against a rubric.

Domain 5: Context management and reliability

You haven't studied this domain yet; use this as a preview, then verify against the course.

1. The context window

KEY POINTS

- Everything counts: system prompt, history, documents, tool definitions, tool results and output.
- ~4 characters per token. Output tokens cost more than input.
- More context isn't better: cost, latency and **quality** all suffer with marginal content.

TERMS

Token	Unit of text, roughly 3/4 of a word.
Context window	Everything the model can see in one call.

2. Summaries and the fact block

KEY POINTS

- Summarisation keeps the gist but **silently drops specifics** (numbers, IDs, names).
- **Extract facts first** into a **persistent fact block** that survives summarisation.
- Update strategies: **append-only** vs **replace on conflict**; prune with **fact aging.**

TERMS

Persistent fact block	Structured prompt region of key facts kept verbatim.
Fact aging	Removing facts not referenced for N turns.

3. Memory, compaction, scratchpads

KEY POINTS

- **Memory tool**: client-side tool reading/writing files under a memory path; persists across conversations.
- **Server-side compaction**: opt-in, summarises when a token threshold is hit (default 150k).
- **Scratchpad** (`notes.md`) holds cross-session state and a decision log; CLAUDE.md wins on conflict.
- Durable facts belong **outside** the transcript.

TERMS

Compaction	Lossy compression of older turns to keep going.
Durable state	Facts stored in files or databases, not just context.

4. Tool-output bloat and trimming

KEY POINTS

- Bloat lowers signal-to-noise and raises context pressure.
- Trim at the **tool wrapper**, not the display layer.
- Strategies: **field filtering** (default), **model summarisation**, **reference indirection.**

TERMS

Field-level filtering	Allow-list of fields kept; rest dropped.
Reference indirection	Store full payload elsewhere, give the agent an ID.

5. Escalation and handoff payload

KEY POINTS

- Triggers must be **explicit**: confidence gate, out-of-policy detection, retry budget exhausted.
- Two failure modes: never escalating (silent failure) and escalating too often.
- Payload: **summary, triggering question (verbatim), what was tried, suggested next step.**
- **Close the loop**: feed the resolution back to the agent.

TERMS

Confidence gate	Escalate when confidence falls below a threshold.
Handoff payload	Structured bundle passed when escalating.

6. Error propagation

KEY POINTS

- Decide deliberately: **local recovery** vs **propagate to coordinator.**
- **Error envelope**: error type, severity (warning / recoverable / fatal), partial results.
- **Idempotency** so retries don't double-write or double-charge.

TERMS

Propagation boundary	Designed line for which errors go up.
Idempotency	Running twice has the same effect as once.

7. Delegating discovery

KEY POINTS

- Verbose searching goes to a subagent with **disposable context.**
- Precise question, defined **return contract**, scoped tools.
- Parent treats subagent output as a **hypothesis to verify.**

TERMS

Return contract	Pre-defined shape of the subagent's answer.

8. Confidence, calibration and review

KEY POINTS

- **Per-field confidence** beats one record score; route only weak fields to humans.
- **Calibrate** on a labeled validation set; read a **reliability diagram.**
- **Stratified sampling** by confidence, category and stakes beats random review.
- Review queue actions: **approve, edit, reject**; log any bypass.

TERMS

Calibration	Stated confidence matches actual accuracy.
Stratified sampling	Oversampling outputs most likely to be wrong.

9. Provenance and conflicts

KEY POINTS

- Every claim maps to a source: **claim-to-source mapping**, **inline citations.**
- Surface source disagreement; resolution: **prefer newer, prefer authoritative, escalate.**
- Avoid the **averaging anti-pattern** (silently blending conflicts).

TERMS

Provenance tracking	Attaching a source pointer to every claim.
Conflict annotation	Showing disagreement instead of hiding it.